

**Universidad de Guadalajara
Centro Universitario de Guadalajara
Licenciatura en Inteligencia Artificial y Ciencia de Datos**



**Especificación de Requerimientos de Software
IEEE 830**

*Sistema de Evaluación de Expresiones Matemáticas
con Algoritmo de Dos Pilas*

Materia: Ingeniería de Software
Proyecto 2. Expresiones matemáticas

Integrantes:

Israel Dueñas Muro
Patricia Jaime Pérez
Ana Elena Macías Amezcua
Edgar Raymundo González Torres
Miguel Angel Jiménez Jiménez

Profesor: Dr. Edgar Gonzalo Cossío Franco
Mayo de 2026

1. Introducción

1.1 Propósito

El propósito de este documento es definir los requerimientos funcionales, no funcionales y técnicos para el desarrollo de un sistema en Python capaz de evaluar expresiones matemáticas mediante el algoritmo de dos pilas. El sistema garantiza el balanceo de signos de agrupación, la persistencia en base de datos y la interoperabilidad mediante formatos estándar (JSON y XML), con acceso controlado por autenticación de usuario y contraseña.

1.2 Alcance

El sistema desarrollado bajo el estándar IEEE 830 comprenderá las siguientes funcionalidades:

- Gestión de Acceso: Autenticación mediante usuario y contraseña.
- Validación de Integridad: Verificación del balanceo de signos de agrupación $()$, $[]$, $\{\}$.
- Captura y Preprocesamiento: El sistema proveerá una interfaz para la entrada de expresiones matemáticas. Antes de la evaluación, el software realizará una normalización de la cadena de entrada eliminando espacios accidentales al inicio y al final mediante la función `strip()`. Asimismo, el sistema deberá ser capaz de segmentar la expresión en tokens (operandos, operadores y signos de agrupación) utilizando el método `split()`, garantizando que múltiples espacios entre caracteres no afecten la integridad del análisis léxico.
- Motor de Evaluación y Visualización: Implementación lógica del algoritmo de dos pilas de Dijkstra para la resolución de expresiones en notación infija. El sistema deberá procesar operandos y operadores de forma independiente mediante estructuras de datos tipo LIFO (*Last-In, First-Out*). De acuerdo con los requerimientos técnicos, el software visualizará en tiempo real (vía consola o interfaz de usuario) el estado dinámico de ambas pilas durante cada etapa de la evaluación. Esta funcionalidad tiene fines estrictamente informativos y de auditoría algorítmica, por lo que no se realizará la persistencia de estos estados intermedios en la base de datos, almacenando únicamente el resultado final obtenido.
- Persistencia de datos: El sistema contará con un módulo de almacenamiento basado en base de datos para registrar de manera permanente cada evaluación realizada. De acuerdo con los acuerdos técnicos, cada registro histórico estará vinculado obligatoriamente al identificador del usuario (ID) autenticado. La información persistida incluirá la expresión matemática original, el resultado numérico final y la marca de tiempo (fecha y hora) de la operación. Se aclara que los estados intermedios del algoritmo (movimientos de las pilas) no serán objeto de persistencia, cumpliendo con el criterio de optimización de almacenamiento acordado con el cliente.
- Interoperabilidad: Exportación e importación en formatos JSON y XML.
- Consulta de historial: Visualización de evaluaciones previas.

1.3 Definiciones y Glosario

- Algoritmo de Dos Pilas (Dijkstra): Método diseñado por Edsger Dijkstra para evaluar expresiones matemáticas en notación infija utilizando dos estructuras de datos tipo pila: una para operandos y otra para operadores.

- Base de Datos (BD): Conjunto de datos pertenecientes a un mismo contexto y almacenados sistemáticamente para su posterior uso. En este sistema, se utiliza para la persistencia del historial de cálculos.
- Interoperabilidad: Capacidad del sistema para intercambiar información con otros programas mediante formatos estándar como JSON (JavaScript Object Notation) y XML (Extensible Markup Language).
- JSON (JavaScript Object Notation): Formato de texto sencillo para el intercambio de datos. Es uno de los formatos utilizados por el sistema para la exportación e importación de registros.
- LIFO (Last-In, First-Out): "Último en entrar, primero en salir". Principio de funcionamiento de una estructura de datos tipo Pila, donde el último elemento agregado es el primero en ser extraído.
- Notación Infija: Forma convencional de escribir expresiones matemáticas donde los operadores se colocan entre los operandos (ejemplo: $5 + 3$).
- Parsing (Análisis Sintáctico): Proceso de analizar una secuencia de tokens para determinar su estructura gramatical con respecto a las reglas matemáticas y de jerarquía.
- Persistencia: Propiedad que permite que la información (historial de cálculos) se conserve de forma permanente después de que el programa se cierra.
- Python: Lenguaje de programación de alto nivel, interpretado y multiparadigma, utilizado para el desarrollo de la lógica del sistema.
- Signos de Agrupación: Caracteres utilizados para alterar la jerarquía de las operaciones; en este sistema se validan paréntesis $()$, corchetes $[]$ y llaves $\{\}$.
- Tokenización: Proceso de análisis léxico donde una cadena de texto (la expresión) se divide en unidades mínimas con significado llamadas "tokens" (números, operadores o signos de agrupación).
- XML (Extensible Markup Language): Lenguaje de marcado que define un conjunto de reglas para la codificación de documentos en un formato que es legible tanto para humanos como para máquinas.

1.4 Referencias

- IEEE Std 830-1998: Práctica recomendada para la especificación de requisitos de software.
- Documentación Oficial de Python 3.x.
- Algoritmo de Dijkstra (Shunting-yard).
- Estándar UML 2.5.
- SWEBOK Guide Version 3.0.

1.5 Apreciación Global

El presente documento está organizado en secciones que detallan el ciclo de vida inicial del proyecto. En la Descripción General se definen las funciones globales y restricciones de formato. En los Requerimientos Específicos se profundiza en la lógica funcional. Las TDCU documentan el comportamiento detallado de cada RF. Finalmente se incluyen el diseño de base de datos y la matriz de casos de prueba.

2. Descripción General

2.1 Perspectiva del Producto

El sistema de Evaluación de Expresiones Matemáticas es un producto de software independiente y autónomo. Su arquitectura está diseñada para operar de forma local, integrando un motor de persistencia robusto que garantiza que la información del usuario no se pierda al cerrar la sesión.

El software interactúa con los siguientes elementos externos:

- Interfaces de Usuario: Una interfaz gráfica o de consola que permite la captura de expresiones y la visualización del proceso algorítmico en tiempo real.
- Interfaces de Software (Base de Datos): El sistema requiere e interactúa obligatoriamente con un Sistema de Gestión de Bases de Datos (SGBD) local (como SQLite). Esta interfaz es responsable de almacenar el historial de cálculos, los resultados y las credenciales de acceso, vinculando cada dato al ID único del usuario.
- Interfaces de Software (Sistema de Archivos): Interactúa con el sistema operativo para gestionar la lectura y escritura de archivos en formatos JSON y XML, permitiendo la portabilidad de los datos almacenados en la base de datos.
- Interfaces de Comunicación: El sistema no requiere conectividad a redes externas, ya que toda la lógica de cálculo y el almacenamiento de datos se ejecutan en el entorno local del usuario.

2.2 Funciones del Producto

El sistema de Evaluación de Expresiones Matemáticas ofrece un conjunto de funcionalidades integradas para el análisis, cálculo y gestión de datos matemáticos. Las funciones principales se resumen a continuación:

- Gestión de Acceso y Seguridad: Provee un mecanismo de autenticación mediante usuario y contraseña para restringir el acceso al sistema y garantizar que cada usuario gestione su propia información de manera privada.
- Validación y Preprocesamiento: El sistema normaliza las expresiones ingresadas (eliminando espacios innecesarios) y verifica la integridad sintáctica, asegurando el correcto balanceo de signos de agrupación ($()$, $[\]$, $\{ \}$) antes de cualquier cálculo.
- Motor de Evaluación Algorítmica: Ejecuta el cálculo de expresiones en notación infija utilizando el algoritmo de dos pilas de Dijkstra. Esta función incluye la capacidad de procesar la jerarquía de operaciones y mostrar al usuario el estado de las pilas en tiempo real.
- Persistencia de Historial: Registro automático en una base de datos local de cada expresión evaluada con su respectivo resultado final y marca de tiempo. Esta función está diseñada para la consulta posterior de resultados por usuario, excluyendo el almacenamiento de estados intermedios del algoritmo para optimizar el rendimiento del sistema.
- Interoperabilidad y Portabilidad: Facilita el intercambio de información mediante módulos de exportación e importación en formatos JSON y XML, permitiendo al usuario respaldar su historial o utilizarlo en aplicaciones externas.

2.3 Características del Usuario

Se han identificado dos perfiles de usuario principales que interactuarán con el sistema de evaluación de expresiones:

- Usuario Estudiante / Final:
 - Nivel Educativo: Estudiantes de nivel medio-superior o superior con conocimientos básicos de aritmética y álgebra.
 - Habilidades Técnicas: Conocimiento básico en el uso de sistemas operativos (Windows/Linux) y manejo de interfaces gráficas o de consola.
 - Experiencia: No requiere experiencia previa en programación, pero debe comprender la jerarquía de operaciones matemáticas para interpretar los resultados y el proceso de las pilas.
- Usuario Administrador / Docente:
 - Nivel Educativo: Profesional con formación en Ciencias de la Computación o Ingeniería.
 - Habilidades Técnicas: Conocimientos avanzados en Estructuras de Datos (Pilas, Colas), Algoritmos de Ordenamiento/Búsqueda y familiaridad con el lenguaje Python.
 - Experiencia: Capacidad para auditar la lógica del algoritmo de Dijkstra y validar la integridad de la base de datos local y los archivos de intercambio (JSON/XML).

2.4 Restricciones del Sistema

El desarrollo y funcionamiento del sistema están sujetos a las siguientes limitaciones:

- Lenguaje de Programación: El sistema debe ser desarrollado exclusivamente en Python 3.10 o superior, utilizando únicamente la biblioteca estándar para la lógica del algoritmo (listas nativas para las pilas).
- Interfaz de Usuario: De acuerdo con el alcance, el sistema operará inicialmente de forma local (Desktop/Consola), no siendo un requisito su despliegue en entornos web o la nube.
- Dependencias de Software: Se requiere que el entorno de ejecución tenga instalado el motor de base de datos SQLite (o el SGBD acordado) para garantizar la persistencia.
- Limitación Matemática: El motor de evaluación solo procesará expresiones en notación infija. No se contempla el soporte para cálculo simbólico (variables), derivadas, integrales o funciones trigonométricas complejas en esta versión.
- Conectividad: El software no requiere ni utilizará conexión a internet. Todas las operaciones, incluyendo la importación y exportación de archivos JSON/XML, se realizarán en el sistema de archivos local.
- Formato de Entrada: El sistema requiere que los tokens de la expresión estén claramente definidos o separables mediante procesos de normalización (strip y split), para evitar ambigüedades en el análisis léxico.

2.5 Supuestos y dependencias

Supuestos:

- Se asume que el usuario final tiene instalada una versión de **Python (3.10+)** compatible con el código fuente proporcionado.

- Se asume que el sistema operativo permite permisos de lectura y escritura en la carpeta del proyecto para la creación y modificación del archivo de la base de datos local y los archivos de exportación (JSON/XML).
- Se asume que las expresiones ingresadas por el usuario siguen las reglas estándar de la aritmética (por ejemplo, no intentar divisiones entre cero, lo cual será manejado como excepción).

Dependencias:

- El sistema depende de la disponibilidad de la biblioteca estándar de Python (módulos sqlite3, json, xml.etree.ElementTree). No se requiere la instalación de librerías de terceros (como Pandas o NumPy), lo que facilita la portabilidad del código.

3. Requerimientos Funcionales

ID	Nombre	Descripción Detallada
RF-01	Captura de expresiones	Entrada de múltiples expresiones matemáticas línea por línea, con separación obligatoria de espacios entre tokens (ej. $(2 + 3)$).
RF-02	Validación algorítmica	Verificación del balanceo de paréntesis $()$, corchetes $[]$ y llaves $\{\}$ para determinar si la expresión es válida antes de procesar.
RF-03	Evaluación con dos pilas	Implementación manual del algoritmo de evaluación utilizando una pila de operandos y una de operadores (sin usar eval()).
RF-04	Persistencia en BD	Guardado automático en base de datos (SQLite/MySQL/PostgreSQL) de: expresión, validez (True/False), resultado y marca de tiempo.
RF-05	Consulta de historial	Visualización de evaluaciones previas mediante una tabla o ventana de consulta que muestre expresión, estado y resultado.
RF-06	Autenticación de acceso	Control de seguridad que permite el ingreso al sistema únicamente mediante usuario y contraseña.
RF-07	Importación y exportación	Capacidad de exportar e importar los registros de las evaluaciones en formatos JSON y XML.
RF-08	Funcionalidades adicionales	Implementación de mejoras: cálculo de potencias $(^)$, soporte para números negativos y visualización del paso a paso de las pilas.

3.2. Requerimientos No Funcionales

ID	Nombre	Nombre	Descripción
RNF-01	Implementación	Lenguaje de desarrollo	El sistema debe ser desarrollado íntegramente en Python 3.10 o superior, utilizando librerías estándar.
RNF-02	Rendimiento	Tiempo de respuesta	La evaluación de expresiones y la actualización de las pilas deben realizarse en un tiempo menor a 1 segundo para garantizar fluidez.
RNF-03	Usabilidad	Simplicidad de Interfaz	La interfaz (ya sea CLI o GUI) debe ser intuitiva, permitiendo que un usuario nuevo realice un cálculo en menos de 3 pasos.
RNF-04	Seguridad	Cifrado de credenciales	Las contraseñas de los usuarios no deben almacenarse en texto plano en la base de datos (se sugiere el uso de hashing).
RNF-05	Portabilidad	Multiplataforma	Compatible con cualquier sistema operativo que soporte Python 3.x.
RNF-06	Fiabilidad	Manejo de excepciones	El sistema debe ser capaz de recuperarse de errores de entrada (como división entre cero) sin cerrarse inesperadamente, informando al usuario.

3.3 Requisitos de Información (Base de Datos)

Este apartado describe los requisitos lógicos y estructurales para toda la información que debe ser persistida en el sistema para dar cumplimiento a los requerimientos funcionales **RF-04**, **RF-05** y **RF-06**.

3.3.1 Requisitos Lógicos

- **Persistencia de Evaluaciones:** El sistema debe registrar cada intento de cálculo, almacenando la expresión original, su estado de validez (booleano), el valor resultante y la marca de tiempo exacta de la operación.

- **Gestión de Identidades:** Se requiere el almacenamiento de credenciales de usuario (nombre de usuario y contraseña protegida mediante hash) para controlar el acceso al historial privado.
- **Integridad Referencial:** Cada registro de evaluación debe estar vinculado de forma obligatoria a un identificador de usuario único (ID), asegurando que el historial sea consultable únicamente por el propietario de la cuenta.
- **Formatos de Intercambio:** El sistema debe ser capaz de transformar los datos almacenados en la base de datos relacional a estructuras jerárquicas en formatos **JSON** y **XML**.

3.3.2 Estructura Técnica (Modelo Relacional)

Se utilizará el motor **SQLite** para implementar las siguientes tablas:

Tabla 1: Usuarios

Campo	Tipo	Descripción
id	INTEGER	Identificador único autoincremental.
usuario	TEXT	Nombre de usuario para autenticación.
password_hash	TEXT	Hash seguro de la contraseña del usuario.

Tabla 2: Historial (Evaluaciones)

Campo	Tipo	Descripción
id_eval	INTEGER	Identificador único autoincremental.
expresion	TEXT	Cadena de la expresión ingresada por el usuario.
valida	BOOLEAN	True si la expresión es válida, False si no lo es.
resultado	TEXT	Valor calculado o mensaje de error.
fecha	DATETIME	Marca de tiempo de la evaluación.
id_usuario	INTEGER	Llave foránea que referencia al usuario que realizó la evaluación.

Anexos

Anexo 1. Protocolo de Entrevista de Elicitación de Requerimientos

Datos de la Sesión

- Entrevistado: Prof. José
- Objetivo: Validar las restricciones técnicas del algoritmo de dos pilas y definir criterios de aceptación para la base de datos y exportación.
- Metodología: Elicitación guiada basada en el estándar IEEE 830.
- Equipo analista: Israel Dueñas, Edgar González, Patricia Jaime, Miguel Jiménez, Ana Elena Macías.
- Fecha: Abril 23 2026.

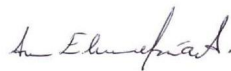
Cuestionario Técnico

Dimensión Técnica	Pregunta del Analista	Respuesta / Acuerdo
Lógica del Algoritmo	¿Debemos desarrollar nuestras propias clases de Pila o es suficiente con usar append() y pop() de listas de Python?	Es suficiente con listas de Python usando append() y pop().
Restricciones de Formato	¿Qué tratamiento debe recibir una expresión con espacios dobles o accidentales al inicio/final?	Se debe aplicar strip() y normalización de espacios antes de tokenizar con split().
Gestión de Errores	Para expresiones inválidas, ¿mensaje descriptivo o código de error?	Mensaje descriptivo (ej. 'Error: División por cero').
Persistencia en BD	¿Es necesario almacenar el ID del usuario que realizó la evaluación?	Sí, el historial debe ser por usuario.
Exportación (XML/JSON)	¿La importación es para re-evaluar expresiones o solo es respaldo?	Solo para reintegrar al historial. Es función de respaldo.
Mejoras Adicionales	¿El paso a paso de las pilas se guarda en BD o solo se muestra en interfaz?	Solo se muestra en interfaz/consola. No se persiste en BD.

Firmas de Conformidad



Prof. José
(Entrevistado)



Equipo de Analistas
(Israel, Edgar, Patricia, Miguel, Ana Elena)

Anexo 2. Tablas Detalladas de Casos de Uso (TDCU)

TDCU-01: RF-01 — Captura de Expresiones

[RF001] – Captura de Expresiones	
1. Datos generales	
Descripción: El sistema deberá permitir ingresar múltiples expresiones matemáticas, una por línea, con separación obligatoria de espacios entre cada token (número, operador, signo de agrupación).	
Actores: Principal: Usuario Secundario: Sistema	
Fecha de elaboración: [28/04/26]	Fecha de última modificación: [05/05/26] Responsable de elaboración: Patricia Jaime Pérez
2. Precondiciones	
El usuario debe tener permisos de escritura/edición El usuario debe estar en el módulo de configuración de fórmulas o parámetros globales. El motor de validación de sintaxis debe estar activo.	
3. Postcondiciones	
La expresión matemática ha sido capturada y almacenada en la memoria temporal del sistema, quedando disponible como entrada para el módulo de validación y evaluación algorítmica.	
4. Flujo normal de eventos (FNE)	
Usuario	Sistema
1. Selecciona el campo de "Ingreso de Expresión". 3. Escribe la expresión (ej. (A + B) / 2). 5. El usuario presiona el botón de "Enviar".	2. Habilita el editor con resaltado de sintaxis. 4. El sistema permite la entrada de texto y aplica el resaltado de sintaxis básico.
5. Alternativas	
Acción	Reacción
2.1.1 El usuario selecciona una expresión desde el historial de cálculos recientes.	2.1.2 El sistema recupera la cadena de texto y la coloca en el campo de "Ingreso de Expresión" para su edición.
6. Excepciones	
Acción	Reacción
3.1.1 El usuario intenta confirmar la expresión con el campo vacío. 3.2.1 El usuario ingresa caracteres no permitidos (ej. letras o símbolos extraños) si tu sistema es solo numérico). 3.3.1 El usuario excede el límite máximo de caracteres permitido para una expresión.	3.1.2 El sistema muestra un mensaje de advertencia: "La expresión no puede estar vacía" y mantiene el enfoque en el editor. 3.2.2 El sistema impide la entrada de dichos caracteres o resalta visualmente que existen caracteres inválidos antes de procesar. 3.3.2 El sistema detiene la escritura y notifica que se ha alcanzado el límite de longitud.

TDCU-02: RF-02 — Validación Algorítmica

[RF002] – [Validación de expresiones]	
1. Datos generales	
Descripción: Se verifica el balanceo de los signos: {}, [] y {}.	
Actores: Principal: Usuario Secundario: Sistema	
Fecha de elaboración: [27/04/26]	Fecha de última modificación: [27/04/26] Responsable de elaboración: Edgar Raymundo González Torres
2. Precondiciones	
El usuario debe de haber ingresado la expresión que será evaluada.	
3. Postcondiciones	
La expresión ingresada quedas validada para proceder a la evaluacion.	
4. Flujo normal de eventos (FNE)	
Usuario	Sistema
	1. Verifica el balanceo de los signos. 2. Confirma que el formato de la expresión es correcta.
5. Alternativas	
Acción	Reacción
1.1.1 Muestra un mensaje diciendo que no se puede calcular la expresón si es invalida.	1.1.2 Vuelve a ingresar la expresión.
6. Excepciones	
Acción	Reacción
1.1	1.2

TDCU-03: RF-03 — Evaluación con Dos Pilas

[RF-03] – [Evaluación con Algoritmo de Dos Pilas]	
1. Datos generales	
Descripción: El sistema implementa manualmente el algoritmo de dos pilas (Shunting-yard de Dijkstra) para evaluar expresiones matemáticas infijas sin usar la función eval() de Python. Actores: Principal: Sistema Secundario: Usuario autenticado Fecha de elaboración: [28/04/26] Fecha de última modificación: [05/05/26] Responsable de elaboración: Israel Dueñas Muro	
2. Precondiciones	
La expresión ha sido validada como correcta por RF-02. La expresión está tokenizada con espacios entre cada elemento.	
3. Postcondiciones	
El sistema obtiene el resultado numérico de la evaluación. El resultado queda disponible para almacenarse (RF-04) y mostrarse al usuario.	
4. Flujo normal de eventos (FNE)	
Usuario	Sistema
1. El usuario ingresa la expresión válida y presiona Evaluar. 3. El usuario visualiza el estado de las pilas en cada paso. 5. El usuario recibe el resultado final.	2. El sistema inicializa las pilas, procesa cada token y aplica el algoritmo de dos pilas. 4. El sistema muestra el estado actualizado de la pila de operandos y operadores. 6. El sistema muestra el resultado o mensaje de error.
5. Alternativas	
Acción	Reacción
2.1 La expresión contiene el operador de potencia (^). 2.2 La expresión contiene números negativos (ej. -3).	2.2 El sistema aplica la precedencia más alta y evaluación de derecha a izquierda para el operador ^. 2.3 El sistema reconoce el signo negativo como parte del número, no como operador binario.
6. Excepciones	
Acción	Reacción
3.1.1 Se intenta dividir entre cero durante la evaluación. 3.2.1 La pila de operandos queda con más de un valor al finalizar.	3.1.2 El sistema captura la excepción y retorna Error: División por cero. 3.2.2 El sistema detecta una expresión malformada y retorna mensaje de error.

TDCU-04: RF-04 — Persistencia en Base de Datos

[RF-04] – [Persistencia en Base de Datos]	
1. Datos generales	
Descripción: El sistema almacena automáticamente cada evaluación realizada en la base de datos.	
Actores: Principal: Sistema Secundario: Base de datos (SQLite)	
Fecha de elaboración: [28/04/26] Fecha de última modificación: [05/05/26] Responsable de elaboración: Israel Dueñas Muro	
2. Precondiciones	
La conexión a la base de datos debe estar activa. La evaluación de la expresión ha concluido.	
3. Postcondiciones	
El registro de la evaluación queda guardado en la base de datos.	
4. Flujo normal de eventos (FNE)	
Usuario	Sistema
1. El usuario confirma la evaluación de la expresión. 3. El usuario visualiza la confirmación de guardado.	2. El sistema almacena la expresión, validez, resultado y fecha/hora en la BD. 4. El sistema muestra notificación de éxito y refresca el historial.
5. Alternativas	
Acción	Reacción
2.1 El usuario solicita no guardar una evaluación específica.	2.2 El sistema omite el paso de persistencia y solo muestra el resultado en pantalla.
6. Excepciones	
Acción	Reacción
3.1.1 La conexión a la BD falla durante el INSERT. 3.2.1 La tabla Evaluaciones no existe en la BD.	3.1.2 El sistema realiza un ROLLBACK y notifica Error al guardar en base de datos. 3.2.2 El sistema crea la tabla automáticamente y reintenta el INSERT.

TDCU-05: RF-05 — Consulta de Historial

[RF005] – Consulta de historial de resultados	
1. Datos generales	
Descripción: El sistema debe permitir al usuario visualizar un registro de las expresiones matemáticas evaluadas previamente. Para cada registro, el sistema debe desplegar: - La expresión original captura. - El estado de validación (si la sintaxis fue correcta o no) - El resultado numérico obtenido de la evaluación. Actores: Principal: Usuario Secundario: Base de datos Fecha de elaboración: [1/05/26] Fecha de última modificación: [1/05/26] Responsable de elaboración: Patricia Jaime Pérez	
2. Precondiciones	
El sistema debe tener conexión activa con la BD. La tabla de resultados en la BD debe estar inicializada.	
3. Postcondiciones	
El usuario cuenta con la información histórica desplegada en pantalla. El sistema mantiene la conexión con la base de datos lista para futuras consultas y no se ha modificado ningún registro previo durante el proceso de visualización.	
4. Flujo normal de eventos (FNE)	
Acción	Reacción
1. El usuario solicita ver el historial (clic en botón/ventana).	2. El Sistema solicita los registros a la Base de Datos.
3. La Base de Datos devuelve la lista de expresiones y resultados.	4. El Sistema formatea los datos y los muestra en una tabla al Usuario.
5. Alternativas	
Acción	Reacción
1.1.1 El usuario decide filtrar el historial por estado (solo ver expresiones "Válidas").	1.1.2 El sistema realiza una consulta filtrada en la base de datos y refresca la tabla mostrando solo los registros coincidentes.
1.2.1.El usuario solicita ver el historial, pero no hay registros en la base de datos.	1.2.2 El sistema despliega un mensaje informativo (ej: "Historial vacío") y muestra la tabla con los encabezados pero sin filas de datos.
1.3.1 El usuario hace clic en una fila de la tabla para copiar un resultado previo.	1.3.2 El sistema carga la expresión seleccionada en el campo de entrada del RF01 (Captura) para permitir una nueva edición o cálculo.
1.4.1 El usuario solicita refrescar la lista manualmente.	1.4.2 El sistema vuelve a conectar con la base de datos y actualiza los datos de la tabla con la información más reciente.
6. Excepciones	
Acción	Reacción
1.2.1 El Usuario intenta filtrar por un criterio que genera un error de sintaxis en la consulta.	1.2.2 El Sistema captura el error, limpia el filtro y muestra: "Criterio de búsqueda no válido".
2.1.1 El Sistema intenta conectar con la Base de Datos pero esta no responde (Timeout/Offline).	2.1.2 El Sistema muestra un mensaje de alerta: "Error de conexión: No se pudo recuperar el historial". El sistema redirige al usuario al menú principal y registra el error en el log
3.1.1 El archivo de base de datos o la tabla están corruptos o inaccesibles.	3.1.2 El Sistema notifica: "Error de integridad de datos" y sugiere contactar a soporte o reiniciar la base de datos.
3.2.1 Memoria insuficiente para cargar un historial extremadamente grande.	3.2.2 El Sistema detiene la carga y muestra: "Error: El historial es demasiado grande para mostrarse completo".

TDCU-06: RF-06 — Autenticación de Acceso

[RF06] - Gestión de Portabilidad de Datos (Exportación/Importación)

1. Datos generales

Descripción:

El sistema debe permitir la persistencia externa y la recuperación de los registros del historial de cálculos mediante archivos de intercambio de datos.

Detalles Funcionales:

RF06.1 Exportación: El sistema debe permitir al usuario exportar la totalidad o una selección del historial de cálculos a archivos locales con formato JSON y XML.

RF06.2 Importación: El sistema debe ser capaz de leer archivos en formato JSON y XML para reintegrar registros previos al historial actual, validando que la estructura de los datos sea compatible con el modelo RegistroResultado.

RF06.3 Integridad: Durante la importación, el sistema debe omitir registros duplicados (basándose en la expresión, fecha y resultado) para mantener la integridad de la base de datos.

Actores:

Principal: Usuario

Secundario: Base de datos

Secundario: Sistema de Archivos (OS)

Fecha de elaboración: [3/05/26]

Fecha de última modificación: 4/05/26]

Responsable de elaboración: Patricia Jaime Pérez

2. Precondiciones

Existencia de Datos: El historial no debe estar vacío. (No tiene sentido exportar un archivo sin registros).

Conectividad Interna: El sistema debe tener acceso activo a la base de datos local para leer los registros.

Permisos de Escritura: El usuario debe tener permisos suficientes en el Sistema Operativo para crear archivos en la ruta de destino.

Disponibilidad del Archivo: El archivo fuente (.json o .xml) debe existir y ser accesible por el sistema.

Formato Correcto: El archivo debe tener una extensión válida y cumplir con la codificación de caracteres esperada (normalmente UTF-8).

Base de Datos Inicializada: El sistema debe tener la estructura de tablas lista para recibir los nuevos datos, incluso si actualmente está vacía.

Espacio en disco: El Sistema Operativo (Actor Secundario) debe garantizar que existe un bloque de espacio libre en el medio de almacenamiento de destino (local o externo) con un tamaño superior al peso estimado del archivo serializado (\$JSON\$ o \$XML\$).

3. Postcondiciones

Persistencia Externa: Existe un nuevo archivo en la ruta especificada por el usuario con la extensión correcta (.json o .xml).

Integridad de Datos: El contenido del archivo coincide exactamente con los registros actuales de la base de datos (sin alteraciones en expresiones o resultados).

Estado de la Aplicación: El sistema regresa a la vista de "Historial" y muestra una notificación de éxito al usuario.

Inmutabilidad Local: La base de datos interna no sufre cambios; exportar es una operación de "solo lectura" para la aplicación.

Sincronización de Base de Datos: Los nuevos registros válidos del archivo han sido insertados en la tabla de la base de datos local.

Actualización de Interfaz: La tabla visual que el usuario ve en pantalla se refresca automáticamente para mostrar los cálculos recién importados.

No Duplicidad: El sistema garantiza que no se insertaron registros idénticos a los ya existentes (evitando redundancia).

Liberación de Recursos: El flujo de lectura del archivo (file stream) se cierra correctamente, liberando la memoria y el acceso al sistema de archivos del OS.

4. Flujo normal de eventos (FNE)

Acción	Reacción
1. El Usuario accede al módulo de "Gestión de Archivos".	2. El Sistema presenta las opciones: "Exportar Historial" e "Importar Historial".
3. El Usuario selecciona "Exportar Historial".	4. El Sistema solicita la elección del formato (<i>JSON</i> o <i>XML</i>).
5. El Usuario elige el formato y el Sistema abre un diálogo de "Guardar como" para definir la ruta de destino.	6. El Sistema solicita al OS verificar que exista espacio suficiente en disco en la ruta elegida para el volumen de datos actual.
7. El Usuario confirma la ubicación de guardado.	8. El Sistema recupera todos los registros de la Base de Datos. 9. El Sistema serializa los datos al formato elegido y escribe el archivo en el Sistema de Archivos (OS).
11. El Usuario selecciona "Importar Historial".	10. El Sistema confirma la exportación exitosa mediante un mensaje informativo.
13. El Usuario elige el archivo y confirma la carga.	12. El Sistema solicita al Usuario que seleccione un archivo fuente del Sistema de Archivos. 14. El Sistema lee el archivo y valida que la estructura cumpla con el modelo de datos y codificación UTF-8. 15. El Sistema filtra registros duplicados comparándolos con la Base de Datos actual. 16. El Sistema inserta los nuevos registros y refresca la tabla visual del historial. 17. El Sistema muestra un resumen de la operación (Ej: "10 importados, 2 duplicados omitidos").

5. Alternativas

Acción	Reacción
5.1.1 El usuario decide cancelar la operación cerrando el cuadro de diálogo de archivos.	5.1.2 El sistema interrumpe el proceso, no realiza ninguna acción de lectura/escritura y regresa al estado inicial del módulo de "Gestión de Archivos".
5.2.1 El usuario selecciona un formato de exportación (<i>JSON</i> o <i>XML</i>) pero el historial de cálculos está actualmente filtrado por una búsqueda específica.	5.2.2 El sistema detiene la carga, muestra un mensaje de "Archivo no compatible" y solicita al usuario verificar la integridad del origen.
14.1.1 El sistema detecta que el archivo seleccionado ya está abierto por otro programa.	14.1.2 El sistema notifica al usuario que el archivo está bloqueado y le solicita cerrarlo o seleccionar uno diferente antes de intentar la carga nuevamente.
15.1.1 El sistema detecta que todos los registros del archivo ya existen en la base de datos local.	15.1.2 El sistema omite la inserción de todos los registros y muestra un resumen indicando: "0 importados, todos los registros estaban duplicados".
15.2.1 El proceso de filtrado detecta que el 100% de los registros ya existen en el sistema.	15.2.2 El sistema finaliza el proceso sin realizar inserciones en la BD y notifica: "No se encontraron registros nuevos para importar".

6. Excepciones

TDCU-07: RF-07 — Importación y Exportación

[RF-07] – [Gestión de Portabilidad de Datos (Exportación/Importación)]	
1. Datos generales	
Descripción: El sistema permite exportar e importar el historial de evaluaciones en formatos JSON y XML. Actores: Principal: Usuario autenticado Secundario: Base de datos, Sistema de Archivos (OS) Fecha de elaboración: [03/05/26] Fecha de última modificación: [05/05/26] Responsable de elaboración: Israel Dueñas Muro	
2. Precondiciones	
El historial no debe estar vacío para exportar. El sistema debe tener conexión activa con la base de datos. El usuario debe tener permisos de escritura en el directorio de destino.	
3. Postcondiciones	
Exportación: Existe un nuevo archivo con la extensión correcta (.json o .xml). Importación: Los nuevos registros válidos han sido insertados en la BD sin duplicados.	
4. Flujo normal de eventos (FNE)	
Usuario	Sistema
1. El Usuario accede al módulo de Gestión de Archivos. 3. El Usuario selecciona Exportar Historial. 5. El Usuario elige el formato y confirma la ubicación de guardado. 11. El Usuario selecciona Importar Historial. 13. El Usuario elige el archivo y confirma la carga.	2. El Sistema presenta las opciones: Exportar Historial e Importar Historial. 4. El Sistema solicita la elección del formato (JSON o XML). 6. El Sistema verifica espacio en disco. 8. El Sistema recupera todos los registros de la BD y los serializa al formato elegido. 9. El Sistema escribe el archivo en el SO. 10. El Sistema confirma la exportación exitosa. 12. El Sistema solicita al Usuario que seleccione un archivo fuente. 14. El Sistema valida la estructura del archivo. 15. El Sistema filtra registros duplicados. 16. El Sistema inserta los nuevos registros y refresca el historial.
5. Alternativas	
Acción	Reacción
5.1.1 El usuario decide cancelar la operación. 14.1.1 El sistema detecta que todos los registros ya existen en la BD.	5.1.2 El sistema interrumpe el proceso sin realizar ninguna acción. 14.1.2 El sistema muestra: 0 importados, todos los registros estaban duplicados.
6. Excepciones	
Acción	Reacción
14.2.1 El contenido del archivo no cumple con el esquema JSON o XML. 15.3.1 Se pierde la conexión con la BD durante la comparación de duplicados.	14.2.2 El sistema muestra Error de formato: Contenido ilegible. 15.3.2 El sistema realiza un Rollback automático y notifica Error crítico de Base de Datos.

TDCU-08: RF-08 — Funcionalidades Adicionales

[RF-08] – [Funcionalidades Adicionales]	
1. Datos generales	
Descripción: El sistema implementa mejoras: soporte para potencia (^), números negativos y visualización del paso a paso de las pilas.	
Actores: Principal: Usuario autenticado Secundario: Sistema	
Fecha de elaboración: [28/04/26] Fecha de última modificación: [05/05/26] Responsable de elaboración: Israel Dueñas Muro	
2. Precondiciones	
El usuario debe estar autenticado. La expresión debe haber pasado la validación de RF-02.	
3. Postcondiciones	
La expresión es evaluada correctamente. El sistema muestra el paso a paso si el usuario lo solicita.	
4. Flujo normal de eventos (FNE)	
Usuario	Sistema
1. El usuario ingresa una expresión con ^ o número negativo y/o activa paso a paso. 3. El usuario visualiza el resultado y el estado de las pilas paso a paso.	2. El sistema reconoce el operador ^ o número negativo y evalúa correctamente. 4. El sistema muestra el estado de ambas pilas después de procesar cada token.
5. Alternativas	
Acción	Reacción
2.1 El usuario usa potencias encadenadas (ej. 2^2^3).	2.2 El sistema evalúa de derecha a izquierda: $2^2(2^3) = 256$.
6. Excepciones	
Acción	Reacción
3.1.1 El operador ^ se usa con un exponente no numérico. 3.2.1 Un número negativo aparece en posición ambigua.	3.1.2 El sistema muestra Operando inválido para potencia. 3.2.2 El sistema solicita al usuario aclarar el formato con paréntesis.